

Five-Stage Pipelined CPU

五級管線處理器專題成果

作者：趙昱琛

專案：github.com/05ChRles09/RV32I-5Stage-Pipeline-CPU

這份專題以 Verilog RTL 建構 RV32I 整數處理器核心，檢視焦點放在指令資料路徑、資料相依處理與控制流程轉向。

5 stages

IF → ID → EX → MEM → WB

32 x 32

Register file ; x0 固定為 0

RV32I

整數指令類別，細節見第 3 頁

專題問題

把一條指令拆成多個階段後，多條指令會同時在處理器內流動。前後指令若讀寫相同暫存器，或分支改變執行路徑，就需要額外的前推、暫停或清除控制。

本專題的重點是把這些控制關係寫進 RTL，並在 testbench 中設計相依、載入後使用及控制轉移等檢查案例。

處理器架構



IF/ID、ID/EX、EX/MEM、MEM/WB 暫存階段資料與控制訊號；指令記憶體由 HEX 程式映像初始化。架構與已完成範圍依專案 README，驗證與 implementation 證據見第 4-5 頁。

本次成果範圍

已完成：RV32I 五級 pipeline、datapath、control unit、hazard detection、forwarding、branch / jump；並附上模擬波形及 Vivado implementation 螢幕截圖。

未列入成果：AXI-Lite、AXI / BRAM、PYNQ-ZU 實板驗證、RV32M MUL、Zbb 與 XDOTP16；皆保留為後續工作。

PIPELINE DESIGN

資料流與冒險處理

Branch / jump 在 EX 階段決定是否轉向；stall、forwarding 與 flush 處理不同類型的相依。



Datapath 與控制



三類冒險的處理方式



RTL IMPLEMENTATION

實作範圍與測試設計

依 GitHub README 的已完成範圍與使用者提供的模擬／implementation 截圖整理；指令細項應以目前 RTL 為最終依據。

RTL 可辨識的指令運算

指令類別	RTL 中的解碼 / 運算	測試覆蓋狀況
R-type	ADD、SUB、AND、OR、XOR、SLL、SRL、SRA、SLT、SLTU	主 testbench 有部分算術與相依案例
I-type ALU	ADDI、ANDI、ORI、XORI、SLLI、SRLI、SRAI、SLTI、SLTIU	opcode / funct 解碼寫於 Control Unit
Branch	BEQ、BNE、BLT、BGE、BLTU、BGEU	另有 signed / unsigned branch 案例
Jump	JAL、JALR；link 資料為 PC+4	主 testbench 與 JALR 專用 testbench
Upper immediate	LUI、AUIPC	LUI 在主 testbench；AUIPC 在分支測試
Load / Store	opcode 層級 load/store；DMEM 以 32-bit word 索引	未見 byte / halfword lane、sign extension 選擇或 misalignment handling

以上為專案 README 所宣告的 RV32I 基本整數指令類別，不代表已通過完整 RISC-V ISA compliance suite。RV32M、Zbb、XDOTP16 在 README 中屬後續規劃，不列為已實作功能。

RTL 模組

rv32i_pipeline.v	頂層 pipeline、PC 更新、stall / flush 接線與輸出觀察點。
control_unit / alu / imm_gen	opcode 控制、整數 ALU 與立即值產生。
forwarding / hazard / branch units	運算元前推、load-use 偵測、分支及跳躍解析。
pipeline regs / regfile / memories	階段暫存器、register file、HEX 指令記憶體與 word data memory。

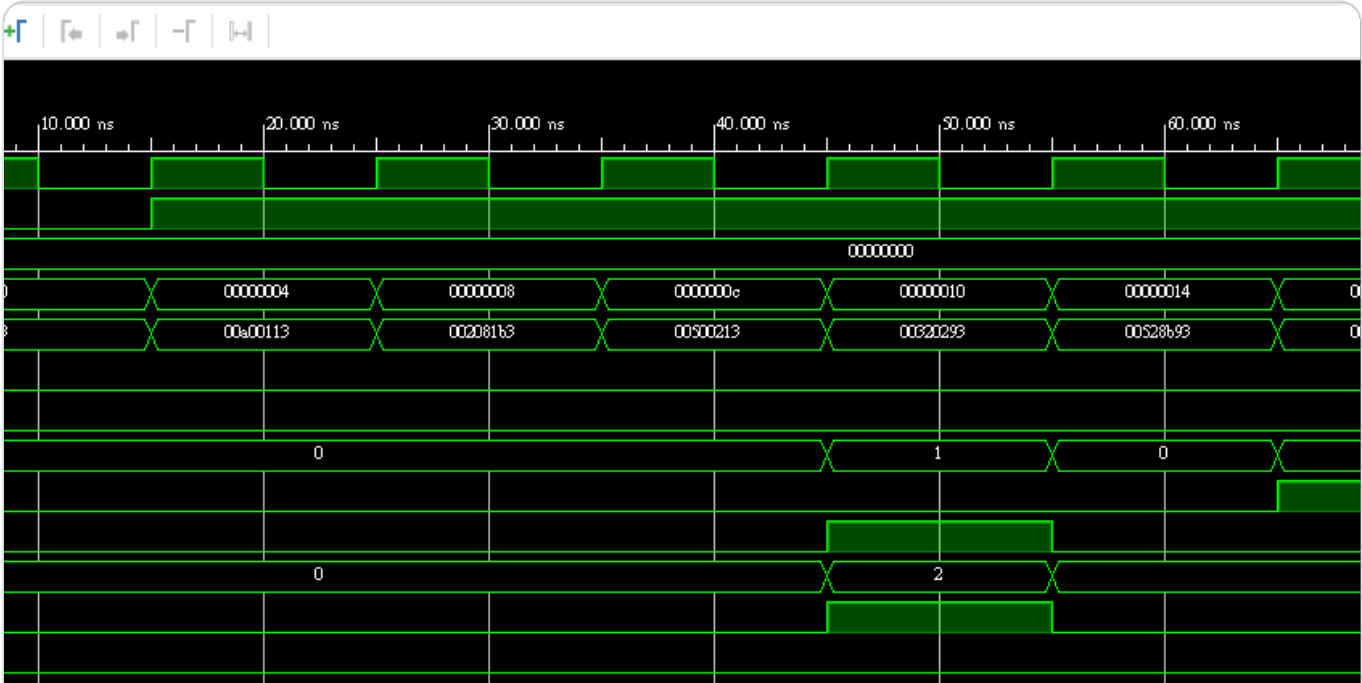
Testbench 設定的檢查案例

tb_rv32i_pipeline	算術與 forwarding chain、sw/lw 與 load-use、taken / not-taken branch、JAL、LUI，並檢查暫存器與記憶體值。
tb_load_use_branch	lw 後緊接 beq；程式檢查 load 值及 branch target 的結果。
tb_jalr	程式檢查 x5 link 預期為 0x0c、跳躍目標指令寫入 x2=7，且 fall-through 不應寫入 99。
tb_extra_branches	程式檢查 BLT、BGE、BLTU、BGEU 與 AUIPC。

SIMULATION EVIDENCE

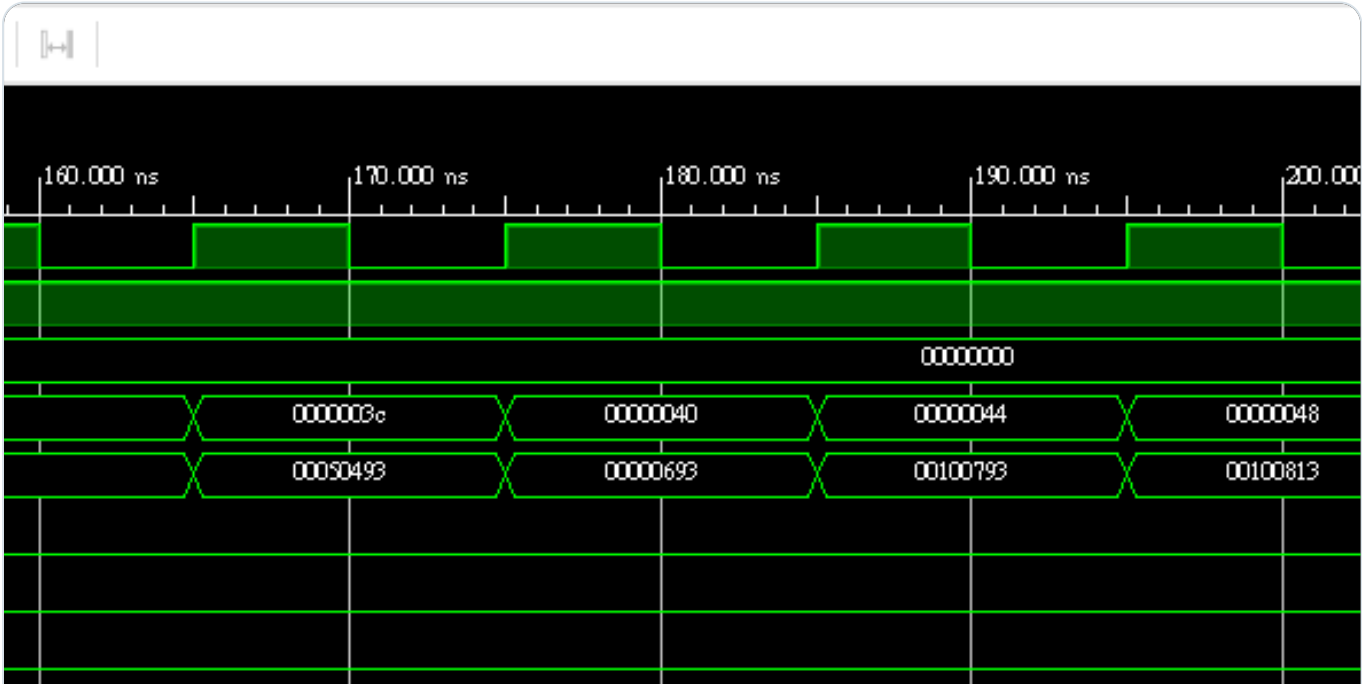
管線控制的波形證據 I

為便於口試或書面審查閱讀，波形改以滿寬放大呈現。以下為基本管線流動與 load-use stall 的驗證情境。



基本五級管線

觀察指令在 IF、ID、EX、MEM、WB 的階段性流動。



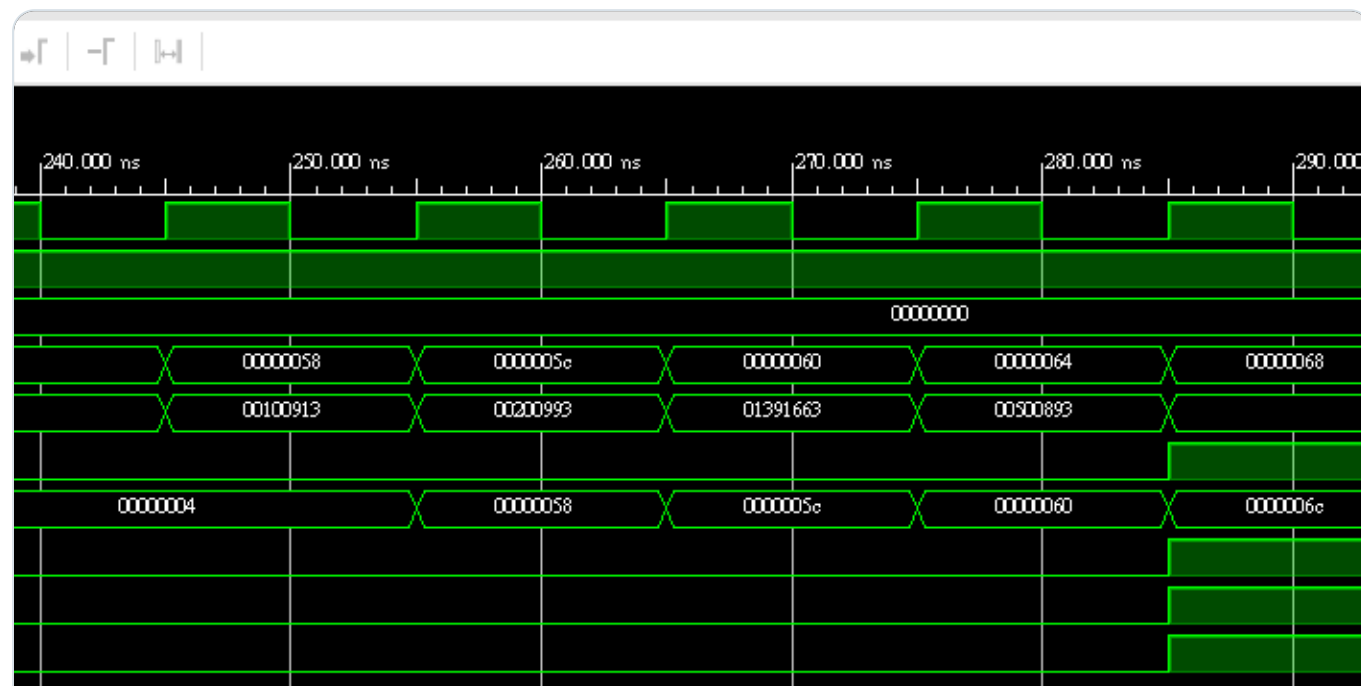
Load-use stall

當 load 結果尚未可供下一條指令使用時，插入 bubble 以維持正確性。

SIMULATION EVIDENCE

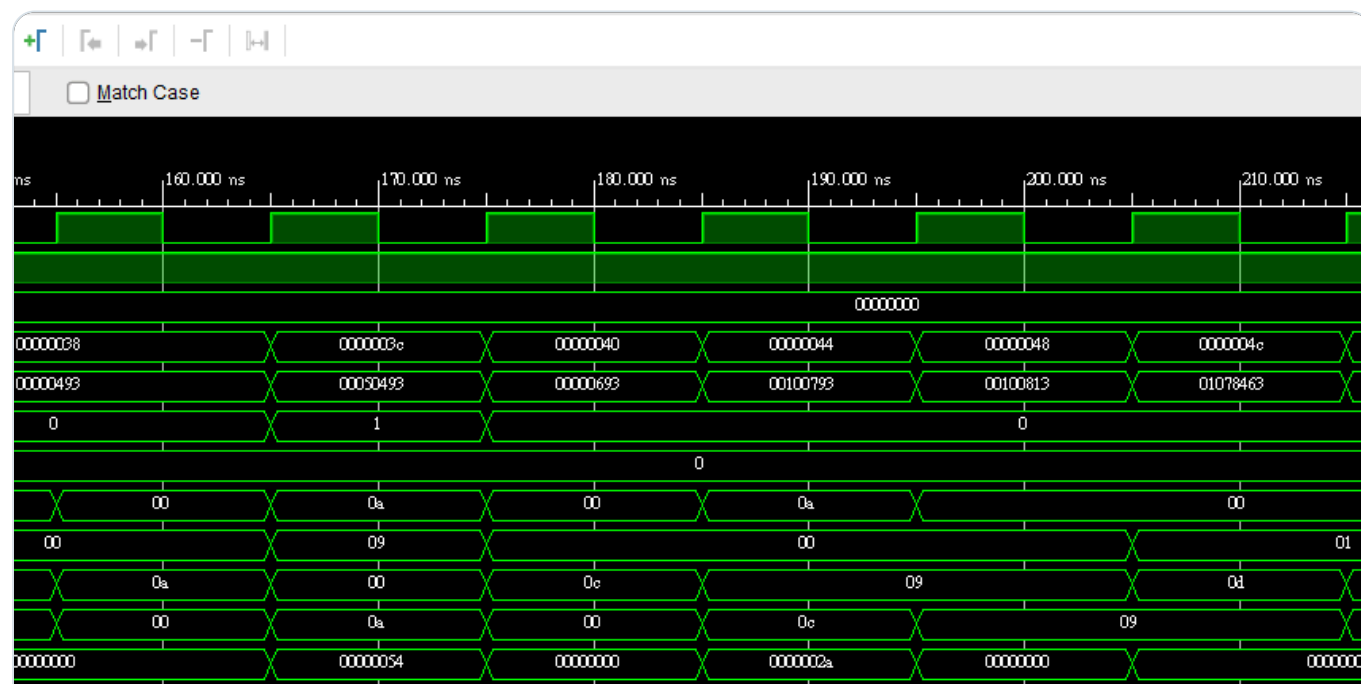
管線控制的波形證據 II

控制流程轉向與資料前推的驗證情境。這兩項機制是維持五級管線正確性與效能的核心。



Branch flush

控制流程改變後，清除錯誤路徑中的管線指令並重新導向 PC。



Data forwarding

將前段計算結果直接提供給 EX stage，降低不必要的停滯。

圖 4-1~5-2 | 波形截圖為驗證案例的視覺證據；詳細 stimulus、訊號命名與通過條件可在後續面試中搭配 testbench 說明。

IMPLEMENTATION RESULT

FPGA 實作結果與延伸方向

提供 Vivado 2022.2 implementation 截圖顯示設計已完成 implementation；畫面同時顯示較新的 implementation 結果可用，因此數值應以重新執行後的最新 report 為準。

4.560 ns

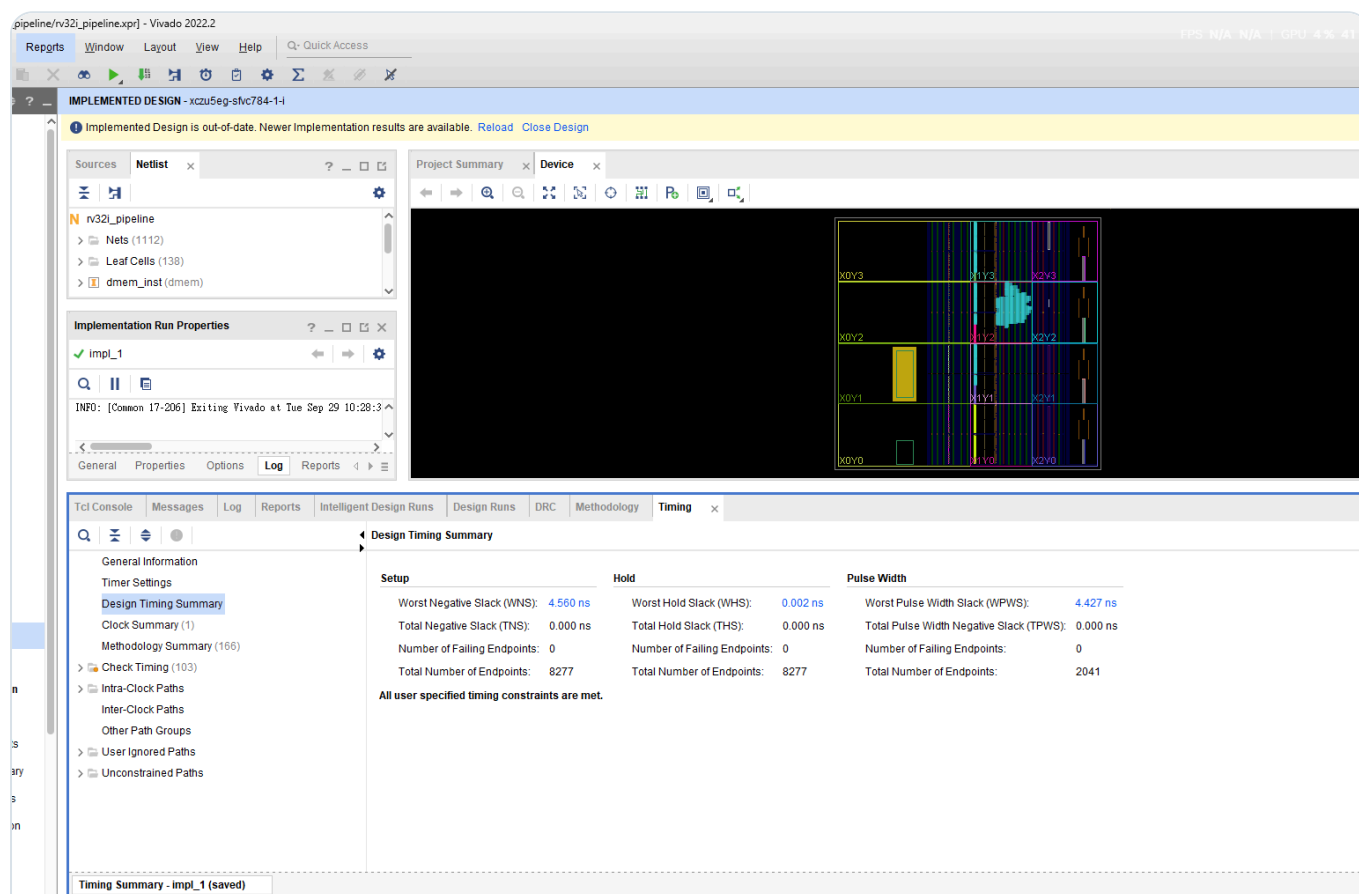
Worst Negative Slack (WNS)
截圖顯示為正值

0.002 ns

Worst Hold Slack (WHS)
截圖顯示為正值

0

Timing failing endpoints
截圖中顯示



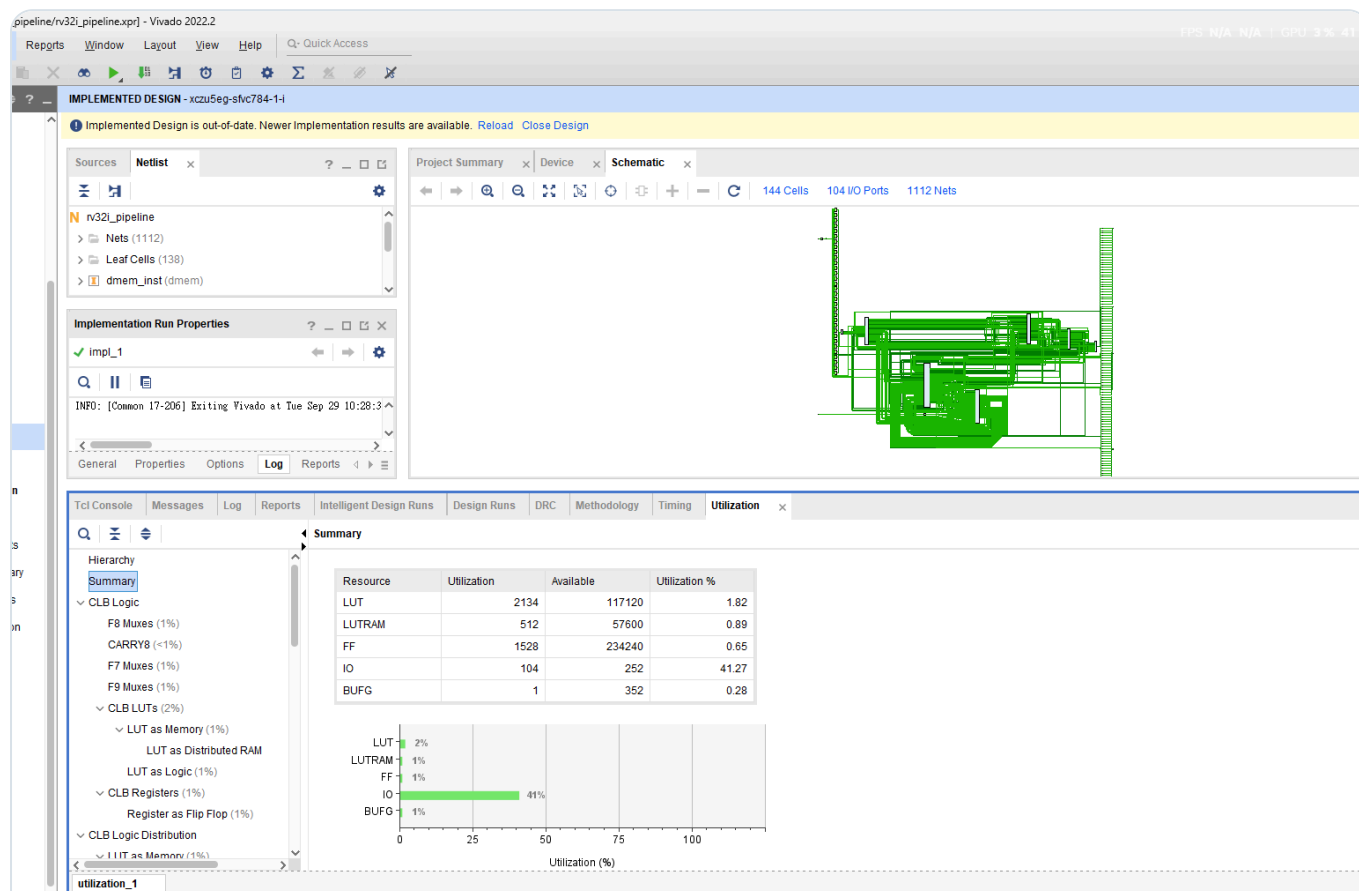
Timing Summary

截圖顯示所有指定 timing constraints 均滿足；口試時可補充 clock constraint 與最新 run 的完整 report。

IMPLEMENTATION RESULT

資源使用與延伸方向

資源使用表格放大呈現，以便清楚檢視 LUT、FF、LUTRAM、I/O 與 BUFG 的使用量及比例。



Utilization Summary

截圖列出 LUT 2,134、FF 1,528、LUTRAM 512、IO 104、BUFG 1 的資源使用情形。

研究所推甄可聚焦的貢獻

已展現能力

以 Verilog / SystemVerilog 實作可合成 CPU 資料路徑；處理 pipeline hazard；以模擬觀察控制行為；完成 Vivado implementation 並解讀 timing 與資源報告。

下一步研究問題

以 AXI-Lite 建立 PS↔PL 控制與除錯介面，再導入 AXI / BRAM；擴充 RV32M MUL、Zbb 與自訂 XDOTP16，並以基準程式比較面積、時脈與 CPI 的取捨。

結語

本專題已完成可合成的 RV32I 五級管線核心，並以波形驗證關鍵 hazard 行為、以 Vivado implementation 取得 timing 與資源證據。下一階段將核心從單一 CPU 設計擴展至可由 PYNQ-ZU PS 控制的 SoC 整合平台。